

Submission Worksheet

Submission Data

Course: IT202-010-S2026

Assignment: IT202 - Milestone 3

Student: Ryan C. (rc728)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 5/6/2026 4:08:00 AM

Updated: 5/7/2026 7:31:14 AM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-3/grading/rc728>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-3/view/rc728>

Instructions

1. Refer to Milestone3 of this doc:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone3 branch
 1. `git checkout Milestone3` (ensure proper starting branch)
 2. `git pull origin Milestone3` (ensure history is up to date)
5. Fill out the worksheet below
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to GitHub
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone3`
 4. On GitHub merge the pull request from Milestone3 to qa
 5. On GitHub create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (3 pts.) Api Data

Progress: 100%

⇒ Task #1 (1 pt.) - Concept of Data Association

Details:

- What is the concept of your data association to users? (examples: favorites, wish list, purchases, assignment, etc)
- Describe with a few sentences

Your Response:

The concept that I chose for my data association to users is watchlist systems that allows the user to save anime that they are interested in watching later. This also allows for multiple people to have one anime in their watchlist and multiple animes to be in one person's watchlist. This association also allows it to make sense for admin functionality with admins being able to add anime to users' watchlists which can act as a recommendation system since it is one of the requirements.



Saved: 5/6/2026 4:43:10 AM

⇒ Task #2 (1 pt.) - Data Updates

Progress: 100%

Details:

- When an associated entity is updated (manually or via API), how is the association affected?
 - Does the user see the old version of the data?
 - Does the user see the new version of the data?
 - Does the user need to have data re-associated or remapped?
- Explain why.

Your Response:

When an associated entity, or anime in my case, is updated either manually or through the api, the watchlist association is not affected. This is because my userWatchlist table only stores the relationship between the user and anime through their IDs so when grabbing it, it will always display the new updated version of the anime or user data. This also allows the user to not need to have data re-associated or remapped since the anime will keep the same ID even if certain fields like title or description are changed.



Saved: 5/6/2026 4:49:41 AM

≡ Task #3 (1 pt.) - Handling Association

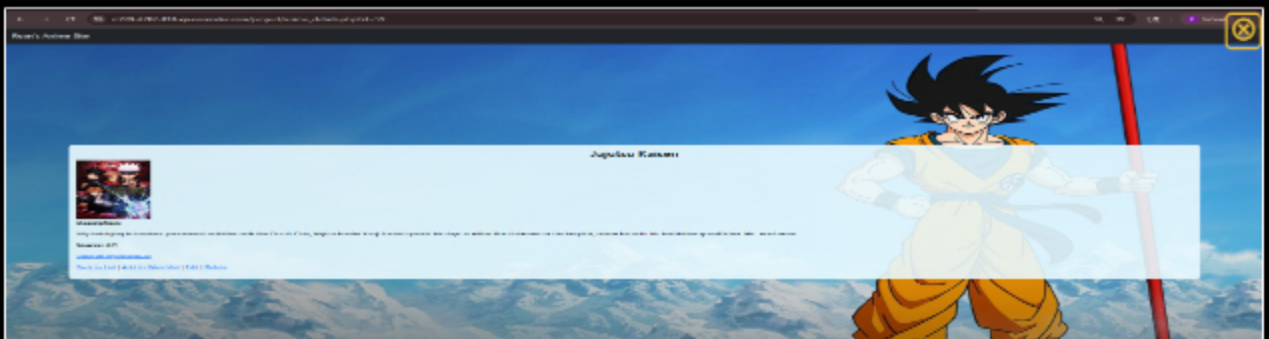
Progress: 100%

Details:

- Show an example page of where a user can get data associated with them
- Ensure the Render.com qa URL is visible
- Caption if this is a user-facing page or admin page
- Include screenshots of code related to this logic including the database query



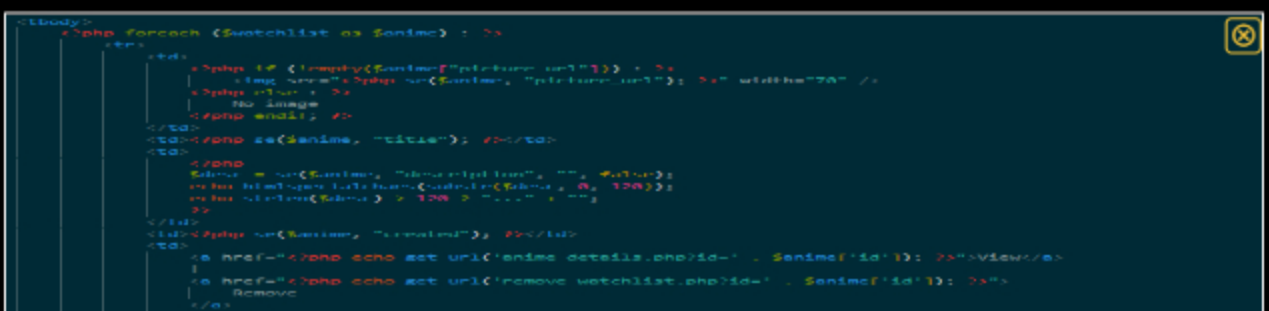
This shows the user-facing page



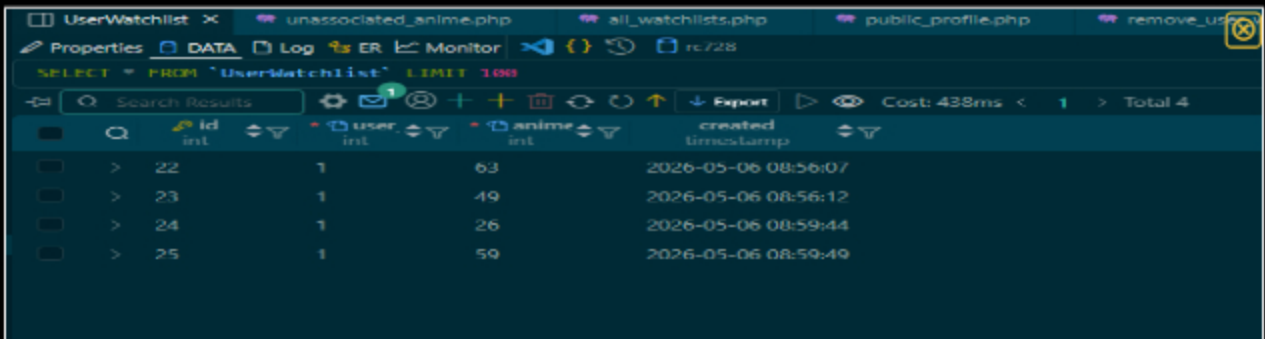
this shows the link to add the anime to the watchlist

```
$query = "SELECT a.id, a.title, a.description, a.picture_url, uw.created
FROM UserWatchlist uw
JOIN Anime a ON uw.anime_id = a.id
WHERE uw.user_id = :user_id";
```

This shows the database query



This shows the display logic



	id int	user int	anime int	created timestamp
>	22	1	63	2026-05-06 08:56:07
>	23	1	49	2026-05-06 08:56:12
>	24	1	26	2026-05-06 08:59:44
>	25	1	59	2026-05-06 08:59:49

This shows the database table showing that the animes were added to the watchlist using only the IDs

 Saved: 5/6/2026 5:17:58 AM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page that triggers and handles this logic

URL #1

https://rc728-it202-010-prod.onrender.com/project/my_watchlist.php



URL

https://rc728-it202-010-prod.onrender.com/project/my_watchlist.php



URL #2

<https://github.com/524RC/RC728-IT202-01057>



URL

<https://github.com/524RC/RC728-IT202-01057>



 Saved: 5/6/2026 5:17:58 AM

Part 3:

Progress: 100%

Details:

- Describe the process of associating data with the user.
- Can it be toggled, or is it applied once?

Your Response:

The process of associating data with the user works by creating a relationship between the user and the anime the user selected in the UserWatchlist table and this works when the user clicks on the add to watchlist button, the system will store the user_id and the anime_id together in the database. The watchlist page then uses an SQL JOIN query to grab all the anime associated with the user through the IDs. The association can be toggled the user have the ability to add and

remove anime from their watchlist whenever or the user can clear their entire watchlist in general.
The association is not permanent and can be modified at any time.



Saved: 5/6/2026 5:17:58 AM

Section #2: (6 pts.) Associations

Progress: 100%

≡ Task #1 (1.50 pts.) - Logged-in User's Associated Entities

Progress: 100%

Details:

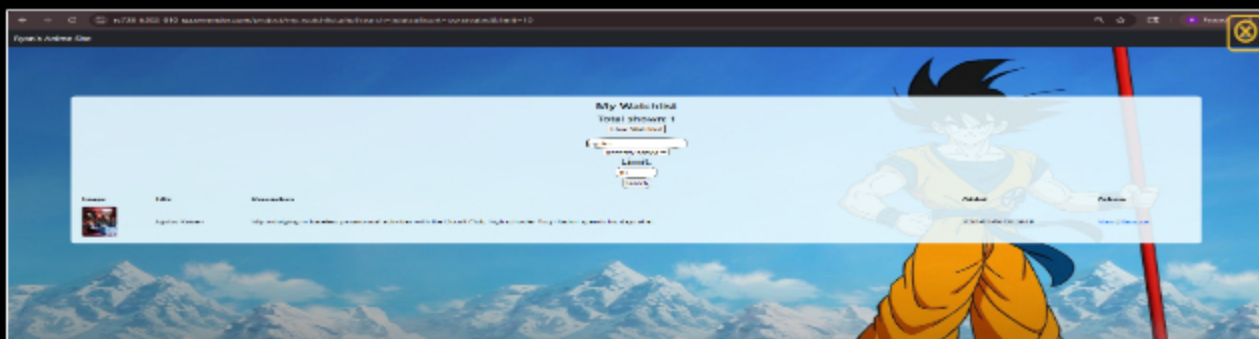
- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a link/button to remove all associations for the particular user
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
 - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
 - A filter with no matching records should show "no results available" or equivalent

🖼 Part 1:

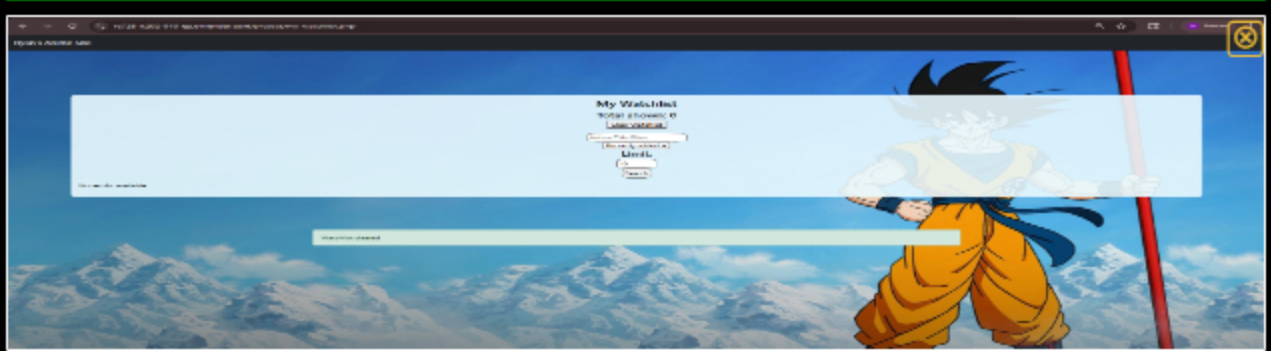
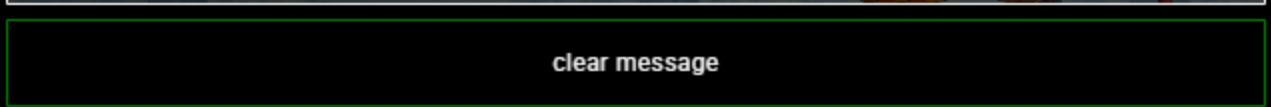
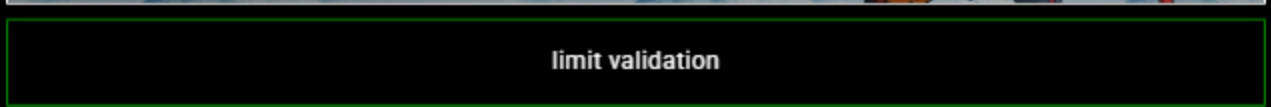
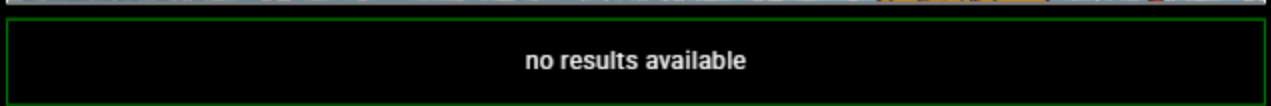
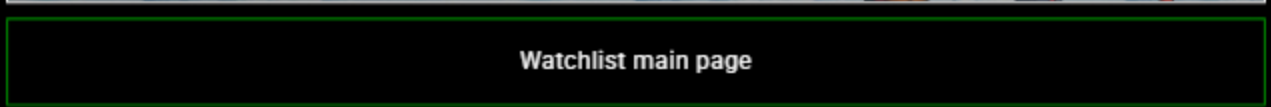
Progress: 100%

Details:

- Show a few examples of this page from Render.com qa with various filters applied
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include the code screenshots related to this page



Filter search



URL #1

https://rc728-it202-010-prod.onrender.com/project/my_watchlist.php



URL

https://rc728-it202-010-prod.onrender.com/project/my_watchlist.php



URL #2

<https://github.com/524RC/RC728-IT202-010>



URL

<https://github.com/524RC/RC728-IT202-010>



Saved: 5/7/2026 5:11:49 AM

⇒ Part 3:

Progress: 100%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

I implemented the association output through my my_watchlist.php page. The page gets the user's ID from the session and uses that ID to query the Userwatchlist table which is why I made it so that there is a login validation because without, the page won't work. The query joins UserWatchlist with the anime table so that the page can display the anime details that are connected to that logged in user allowing the display of the images, titles, descriptions, and the dates added. The watchlist works very similarly to the anime list page where each line item displays the summary of the anime in the table that's associated with the user. The difference between the animelist and the watchlist is the action links at the end of each line of the anime. These action links are view and remove. The view link is like the animelist page where it brings the user to the anime details page. The remove link works by only deleting the watchlist relationship and not the anime entirely so the anime still exists. I also have a clear watchlist button outside of the list which basically clears the whole watchlists of all its associations for the logged in user. Other features outside the table are the filters which is handled through the title search fields, there's sorting which can either sort by recently added or title, then there's the limit field which lets the user choose how many records are shown which the server checks if the limit is between 1 and 100. Also if there is no matching records found then the page will display a no result available message. The page also includes a result stat which shows the total number of results that are currently displayed.



Saved: 5/7/2026 5:11:49 AM

≡ Task #2 (1.50 pts.) - All Users Association Page

Progress: 100%

Details:

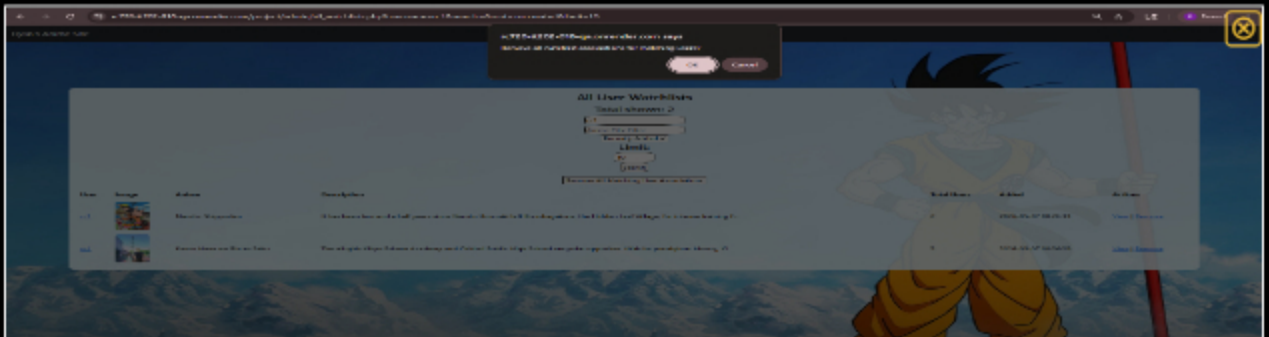
- Each line item should have a logical summary
- Each line item should include the username this entity is associated with

Clicking the username should redirect to that user's public profile

the username filter applied



No results available



remove matching associations



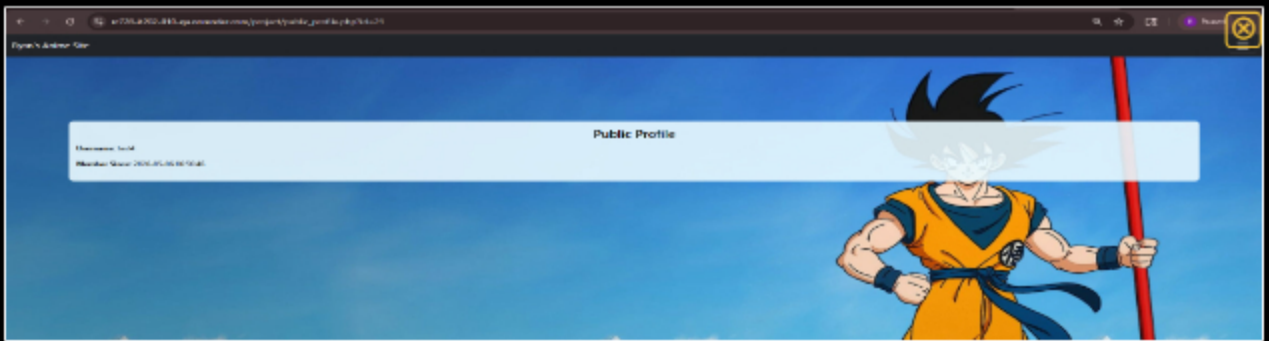
after removing matching associations



remove single relationship



after removing single relationship



public profile



limit validation

```
if (!has_role("Admin")) {  
    flash("No permission", "danger");  
    die(header("Location: " . get_url("landing.php")));  
}
```

admin validation

```
$username = se($_GET, "username", "", false);  
$search = se($_GET, "search", "", false);  
$sort = se($_GET, "sort", "uw.created", false);  
$limit = (int)se($_GET, "limit", 10, false);  
  
if ($limit < 1 || $limit > 100) {  
    $limit = 10;  
}
```

filter, sort, and limit setup

```
$query = "SELECT  
    uw.id as watchlist_id,  
    Users.id as user_id,  
    Users.username,  
    a.id as anime_id,  
    a.title,  
    a.description,  
    a.picture_url,  
    uw.created,
```

SQL JOIN query used to associate users with anime watchlists

username and title filtering and sorting logic

remove matching associations form

single relationship delete logic for remove_user_watchlist.php

remove matching associations logic for remove_matching_watchlist.php

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1

https://rc728-it202-010-prod.onrender.com/project/admin/all_watchlists.php



URL

https://rc728-it202-010-prod.onrender.com/project/admin/all_watchlists.php



URL #2

<https://github.com/524RC/RC728-IT202-01057>



URL

<https://github.com/524RC/RC728>



URL #3

<https://github.com/524RC/RC728-IT202-01059>



URL

<https://github.com/524RC/RC728>



Part 3:

Progress: 100%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

I implemented the association output through my all_watchlist.php admin page. This page queries the UserWatchlist table and it joins it with the Users and Anime tables which allows it to display on each row which user is associated with which anime. Like the rest of the lists, it also displays the all the anime details along, but also includes the user that item is associated with. How i solves the requirements is by making the username a link to the user's public profile which i had to make. similarly to the regular watchlist, i included a view link which opens to the anime details page, and added a remove link which deletes only the watchlist relationship and not the user or anime. Outside the table, the page includes a total shown stat, username filtering, anime title filtering, sorting options, and a limit field which lets the user choose how many records are shown which the server checks if the limit is between 1 and 100. It also shows a message of no results available if there were no matching results. Finally there is a button to remove all watchlist associations for users matching the username filter.

☰ Task #3 (1.50 pts.) - Unassociated Page

Progress: 100%

Details:

- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
 - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
 - A filter with no matching records should show "no results available" or equivalent

🖼️ Part 1:

Progress: 100%

Details:

- Show a few examples of this page from Render.com qa with various filters applied
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query



this shows the unassociated page



this shows the filter applied




```
$query = "SELECT id, title, description, picture_url, is_api, created, modified
FROM Anime
WHERE id NOT IN (
    SELECT anime_id FROM UserWatchlist
)";
```

SQL query grabbing anime with no watchlist associations

```
if (!empty($search)) {
    $query .= " AND title LIKE :search";
    $params[":search"] = "%$search%";
}
```

anime title filtering logic

```
if ($sort === "title") {
    $query .= " ORDER BY title ASC LIMIT $limit";
} else {
    $query .= " ORDER BY $sort DESC LIMIT $limit";
}
```

total shown stat display

```
<?php if (empty($anime_list)) : ?>
    <p>No results available</p>
<?php else : ?>
    <table class="anime-table">
```

no result available message

```
<?php foreach ($anime_list as $anime) : ?>
    <tr>
        <td>
            <?php if (empty($anime['picture_url'])) : ?>
                <img alt="Anime placeholder image" />
            <?php else : ?>
                <img alt="$anime[picture_url]" />
            </td>
            <td>
                <?php echo($anime['title']);
            </td>
            <td>
                <?php echo($anime['description']);
            </td>
            <td>
                <?php echo($anime['created']);
            </td>
            <td>
                <?php echo($anime['modified']);
            </td>
        </tr>
    </foreach>
```


☰ Task #4 (1.50 pts.) - Admin Association Page (Like User Roles)

Progress: 100%

Details:

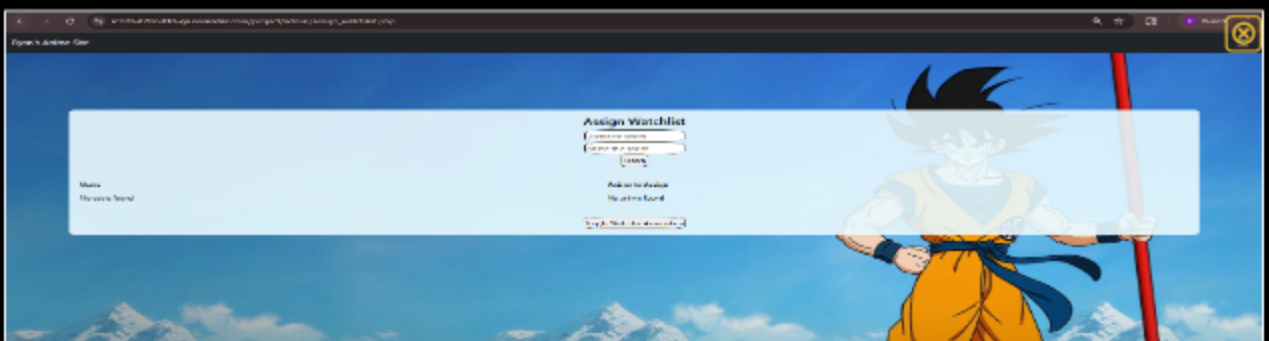
- The page should have a form with two fields
 - Partial match for username
 - Partial match for entity reference (name or something user-friendly)
- Submitting the form should give up to 25 matches of each
 - Likely best to show as two separate columns
- Each entity and user will have a checkbox next to them
- Submitting the checked associations should apply the association if it doesn't exist; otherwise it should remove the association
 - A filter with no matching records should show "no results available" or equivalent

🖼 Part 1:

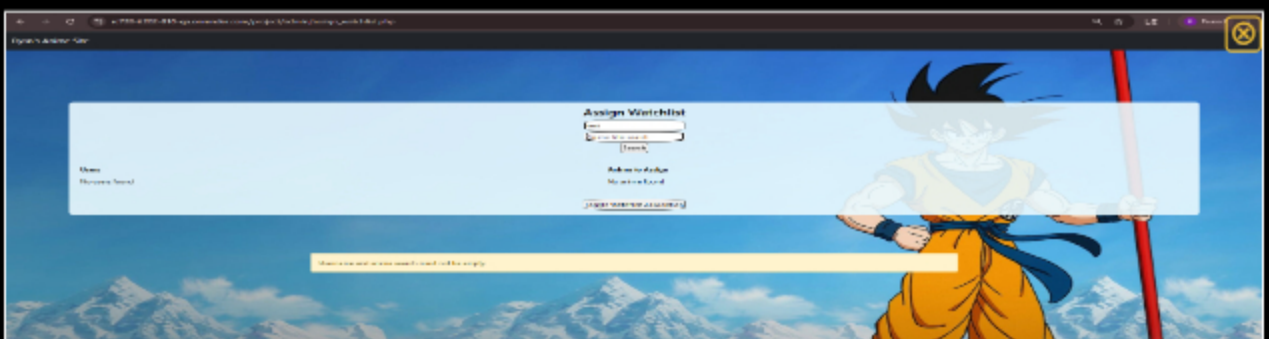
Progress: 100%

Details:

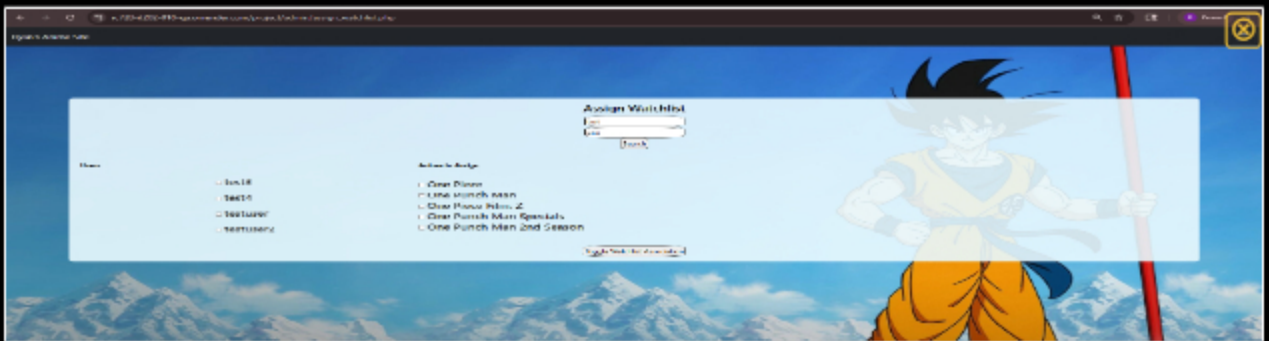
- Show a few examples of this page from Render.com qa with various selections having been submitted
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query



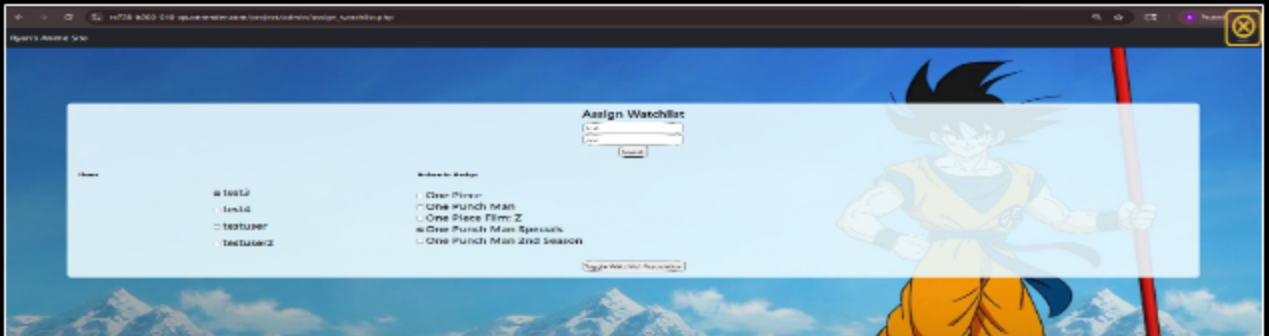
the assign watchlist page along with no user found



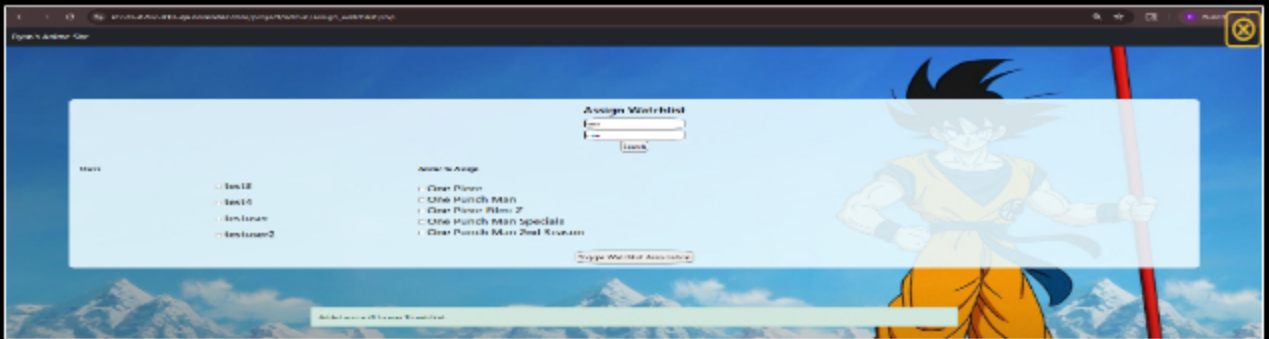
validation message for empty filter



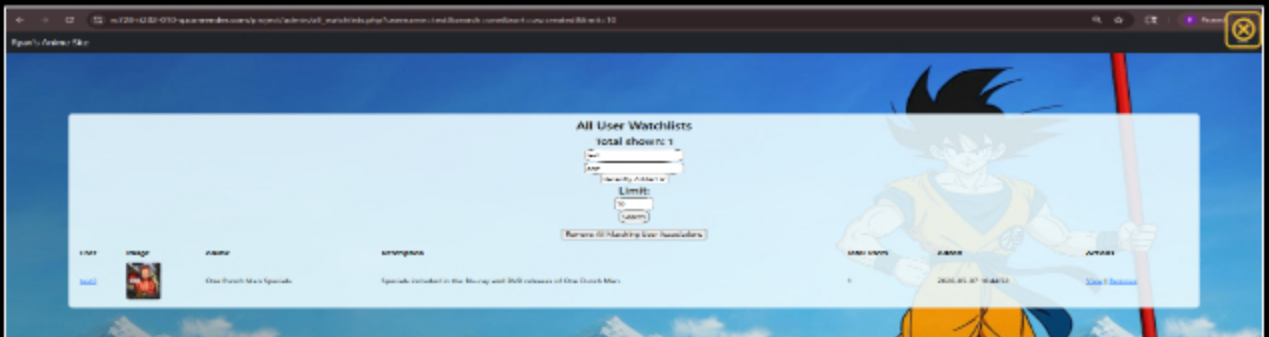
user and title search working



before associated successfully added



after association successfully added



shows that the association was actually added

```
if (!has_role("Admin")) {  
    flash("You don't have permission to view this page", "warning");  
    die(header("Location: " . get_url("landing.php")));  
}
```

admin validation

```
if (isset($_POST['username'])) {
    $username = $_POST['username'];
    $password = $_POST['password'];

    if (empty($username) || empty($password)) {
        $message = "Please enter a valid username and password";
    } else {
        $username = trim($username);
        $password = trim($password);

        $stmt = $db->prepare("SELECT * FROM users WHERE username = :username AND password = :password");
        $stmt->execute();
        $result = $stmt->fetch(PDO::FETCH_ASSOC);

        if ($result) {
            session_start();
            $_SESSION['username'] = $username;
            $_SESSION['password'] = $password;
            header("Location: index.php");
            exit();
        } else {
            $message = "Invalid username or password";
        }
    }
}

if (isset($_POST['username'])) {
    $username = $_POST['username'];
    $password = $_POST['password'];

    if (empty($username) || empty($password)) {
        $message = "Please enter a valid username and password";
    } else {
        $username = trim($username);
        $password = trim($password);

        $stmt = $db->prepare("SELECT * FROM users WHERE username = :username AND password = :password");
        $stmt->execute();
        $result = $stmt->fetch(PDO::FETCH_ASSOC);

        if ($result) {
            session_start();
            $_SESSION['username'] = $username;
            $_SESSION['password'] = $password;
            header("Location: index.php");
            exit();
        } else {
            $message = "Invalid username or password";
        }
    }
}
```

assign and deletion logic

```
<form method="POST">
    <input type="search" name="username" placeholder="Username search" value="<?php echo htmlspecialchars($username); ?>" />
    <input type="search" name="anime_search" placeholder="Anime title search" value="<?php echo htmlspecialchars($anime_search); ?>" />
    <input type="submit" value="Search" />
</form>
```

search form for usernames and anime titles

```
<?php
// Search for usernames and anime titles
if (isset($_POST['username'])) {
    $username = $_POST['username'];
    $stmt = $db->prepare("SELECT * FROM users WHERE username = :username");
    $stmt->execute();
    $result = $stmt->fetch(PDO::FETCH_ASSOC);

    if ($result) {
        echo "User found: " . $result['username'] . "  
";
    } else {
        echo "User not found.  
";
    }
}

if (isset($_POST['anime_search'])) {
    $anime_search = $_POST['anime_search'];
    $stmt = $db->prepare("SELECT * FROM anime WHERE title = :anime_search");
    $stmt->execute();
    $result = $stmt->fetch(PDO::FETCH_ASSOC);

    if ($result) {
        echo "Anime found: " . $result['title'] . "  
";
    } else {
        echo "Anime not found.  
";
    }
}
```

checkbox output for users and anime associations

 Saved: 5/7/2026 7:14:21 AM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

<https://rc728-it202-010->

prod.onrender.com/project/admin/assign_watchlist.php

URL #2

<https://github.com/524RC/RC728->

IT202-01059



URL

<https://github.com/524RC/RC728>



Saved: 5/7/2026 7:14:21 AM

Part 3:

Progress: 100%

Details:

- Describe how your code solves the requirements; be clear

Your Response:

for the admin association page i made a assign watchlist page through assign_watchlist.php admin page. The page contains two search fields, one for anime and one for usernames. when the form is submitted, the page searches both the Users and Anime tables using partial matching and displays up to 25 matching results for each section. Then the users and anime are displayed in two separate columns with checkmarks next to them similar to how the role assign page works. when the toggle watchlist association button is pressed, the code first checks if the user and the anime selected are already associated in the UserWatchlist table. if it doesn't exist in the table, then page adds the relationship, but if it does already exist, then the page removes the relationship. This allows the admin to add and remove watchlist associations. The page also includes validation and feedback messages like how it shows a warning if the search field is empty or how it displays a message if there are no matching user or anime found. This page is also admin access only.



Saved: 5/7/2026 7:14:21 AM

Section #3: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - GitHub Details

Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history

added anime watchlist and the baselinefile for all_watchlists #57

Code



Merged 5286C merged 1 commit into from PR #57 yesterday

Conversation Commits Checks Files changed

+0/-0



This shows the commits tab



this shows the commits tab

 Saved: 5/7/2026 7:16:20 AM

Part 2:

Progress: 100%

Details:

Include the link to the pull request for Milestone3 to qa (should end in `/pull/#`)

URL #1

<https://github.com/524RC/RC728-IT202401157>



URL

<https://github.com/524RC/RC728>



URL #2

<https://github.com/524RC/RC728-IT202401159>



URL

<https://github.com/524RC/RC728>



 Saved: 5/7/2026 7:16:20 AM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment I would say was making the myAnimeWatchlist page or the assign watchlist page. I reused a lot of code to make these pages so all i had to do was tweak the code a little bit for it to work for the page. Making the public profile wasn't that hard either.



Saved: 5/7/2026 7:27:26 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was making the all watchlist. It had me thinking for a little bit on how to grab the user associated with the anime and all the features needed for the page had me working on it and tweaking it for a while.



Saved: 5/7/2026 7:31:14 AM